MORROW

PAT 4 Master Study Guide

Know the architecture. Explain the code. Run the demo.

Java 21 • Spring Boot 3.5.16 • Spring JDBC • Oracle XE 11g

CORE MESSAGE
The browser talks to Spring Boot over HTTP. Controllers route requests. Services enforce rules. JdbcTemplate executes parameterized SQL through the Oracle JDBC driver. Oracle stores users, wallets, transactions and ledger rows.

This guide matches the current Morrow project using the simplified table names USERS, WALLETS, TRANSACTIONS and LEDGER. It is written for your actual source layout, not a generic Spring project.

Exam strategy: understand the end-to-end flow first; then learn the exact files and commands. You do not need to memorize every CSS rule.

1. Tonight's study plan

Time	Study target	What you must be able to do
25 min	Architecture + tech stack	Draw the four layers and explain each technology.
35 min	JDBC + Oracle connection	Point to pom.xml, application.properties, JdbcTemplate and the JDBC URL.
35 min	Database schema + SQL	Explain all four tables, primary/foreign keys, sequences/triggers and DDL/DML.
40 min	Authentication + sessions	Talk through registration, login, BCrypt, JSESSIONID and logout.
40 min	Top-up + transfer	Explain @Transactional, FOR UPDATE, balance checks, transactions and ledger.
25 min	Live demo + viva	Run the project, execute key SELECTs and rehearse questions aloud.

The 60-second project explanation

"Morrow is a digital wallet management web application for my PAT 4 project. It uses Java 21 and Spring Boot 3.5.16 for the backend, Thymeleaf to serve the HTML page, and HTML/CSS/vanilla JavaScript for the interface. The frontend calls REST-style endpoints. Controllers validate and route those requests to the service layer. The service layer applies the wallet and authentication rules and uses Spring JdbcTemplate to run parameterized SQL. The Oracle JDBC Thin driver connects the application to Oracle Database XE 11g, SID XE on port 1521. Oracle persists the users, wallets, transactions and ledger. Passwords are stored as BCrypt hashes, and transfers update both wallets plus transaction/ledger records inside a database transaction. All funds are simulated; no bank is connected."

2. Architecture: explain the request path

Layer / file	Responsibility	Morrow example
Browser / frontend	Displays UI and sends HTTP requests.	index.html, app.css, app.js; JavaScript fetches /api/auth/login.
Web/controller layer	Maps URL + HTTP method to Java method; handles request/response.	AuthController, WalletController, HomeController, HealthController.
Service layer	Business rules and coordinated data operations.	AuthService, WalletService.
JDBC/data access	Parameterized SQL, result mapping, connection/exception handling.	JdbcTemplate inside AuthService and WalletService.
Oracle XE 11g	Durable relational storage and constraints.	USERS, WALLETS, TRANSACTIONS, LEDGER.

Important accuracy note: there is no separate repository/DAO package in this code. The service classes use JdbcTemplate directly. If asked for the layers, say controller → service → JdbcTemplate/DataSource → Oracle, not "repository classes" that do not exist.

What does “full round trip” mean?

1. A user clicks Sign in; app.js sends a POST request with JSON to /api/auth/login.
2. AuthController receives the request and calls AuthService.login().
3. AuthService queries USERS through JdbcTemplate and verifies the BCrypt password hash.
4. AuthController establishes the HTTP session and returns a wallet summary JSON response.
5. JavaScript receives the response and paints the user name and balance into the dashboard.

3. Technologies and why we use them

Technology	What it is / why Morrow uses it
Java 21	Backend programming language and runtime.
Spring Boot 3.5.16	Bootstraps the application and auto-configures web server, DataSource, JDBC and other beans.
Spring MVC	Maps HTTP routes to controller methods.
Thymeleaf	Template engine; HomeController returns the index template.
HTML/CSS/JavaScript	Structure, visual styling, and browser interactions/API calls.
Spring JDBC / JdbcTemplate	Executes SQL with bound parameters and maps results; reduces repetitive JDBC boilerplate.
Oracle JDBC Thin driver	Implements JDBC communication with Oracle over TCP using a Java driver.
Oracle XE 11g	Relational database that permanently stores the project data.
Maven	Downloads dependencies, compiles, tests, packages and runs the project.
BCryptPasswordEncoder	Hashes new passwords and checks submitted passwords against stored hashes.
HikariCP	Connection pool automatically used by Spring Boot’s DataSource setup.

JDBC versus ODBC versus JPA

- JDBC = Java Database Connectivity: Java’s standard API for connecting to and executing SQL against a database. Morrow uses JDBC via Spring JdbcTemplate.
- ODBC = Open Database Connectivity: a language-independent/native database connectivity standard. Morrow does not use ODBC.
- JPA/Hibernate = ORM approach mapping Java objects to tables. Morrow does not use JPA/Hibernate; its SQL is explicit and visible in Java.

4. Know these exact files

File / folder	What to point at in the viva
pom.xml	Project metadata, Java version, Spring Boot parent, JDBC starter and Oracle driver dependencies.
src/main/resources/application.properties	Oracle JDBC URL, database username/password environment variables, driver class, port, session settings and connection pool settings.
.../MorrowWalletApplication.java	main() and @SpringBootApplication; starts Spring Boot.
.../auth/AuthController.java	Registration/login/me/logout HTTP endpoints and session setup.
.../auth/AuthService.java	BCrypt hashing, account lookup, inserts into USERS and initial WALLETS.
.../auth/AuthRequests.java	Register/Login request records and bean validation annotations.
.../auth/SessionUser.java	Session attributes and requirement for a signed-in session.
.../wallet/WalletController.java	Summary, transaction history, top-up and transfer endpoints.
.../wallet/WalletService.java	The main JDBC and wallet business logic; balance updates, FOR UPDATE, transaction/ledger inserts.
.../wallet/WalletRequests.java	Top-up and transfer input validation.
.../web/HomeController.java	Returns the Thymeleaf page named index for GET /.
.../web/HealthController.java	/health and /health/db connectivity checks.
.../api/ApiExceptionHandler.java	Converts validation, API and database failures to safe HTTP JSON errors.
.../web/SecurityHeadersFilter.java	Adds common browser security headers.
src/main/resources/templates/index.html	HTML and IDs used by JavaScript. Includes Thymeleaf asset paths.
src/main/resources/static/js/app.js	Fetch helper and browser logic; calls API endpoints.
src/main/resources/static/css/app.css	Visual appearance; CSS only should not control database logic.
database/01_schema.sql	DDL: creates tables, keys, constraints, indexes, sequences and triggers.
database/02_sample_dml.sql	Optional PL/SQL/DML example for simulated SQL*Plus top-up.
database/03_useful_queries.sql	Queries used to inspect the tables.
database/05_rename_existing_tables.sql	One-time in-place rename from legacy MW_* table names to simpler table names.

5. The JDBC connection: the exact pieces

A. Dependency in pom.xml

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-jdbc</artifactId>
</dependency>
<dependency>
  <groupId>com.oracle.database.jdbc</groupId>
  <artifactId>ojdbc8</artifactId>
  <version>${oracle.jdbc.version}</version>
</dependency>
```

The JDBC starter gives Spring JDBC and support for DataSource auto-configuration. The Oracle driver dependency provides the vendor-specific driver. In this specific project POM, oracle.jdbc.version is 12.2.0.1 and the artifact is ojdbc8. Do not claim that this current ZIP uses ojdbc11. This is a legacy driver line because the school machine uses Oracle XE 11g.

B. Configuration in application.properties

```
server.port=${SERVER_PORT:8080}
spring.datasource.url=${ORACLE_JDBC_URL:jdbc:oracle:thin:@localhost:1521:XE}
spring.datasource.username=${ORACLE_DB_USER:MORROW_APP}
spring.datasource.password=${ORACLE_DB_PASSWORD:change_me}
spring.datasource.driver-class-name=oracle.jdbc.OracleDriver
spring.datasource.hikari.maximum-pool-size=5
spring.datasource.hikari.connection-timeout=5000
server.servlet.session.timeout=30m
server.servlet.session.cookie.http-only=true
server.servlet.session.cookie.same-site=lax
```

Environment placeholders mean “use the environment variable if supplied; otherwise use the value after the colon.” The change_me password is only a fallback/example and should not be relied on for a real setup.

C. Actual JDBC use in Java

```
import org.springframework.jdbc.core.JdbcTemplate;

private final JdbcTemplate jdbc;

public AuthService(JdbcTemplate jdbc) {
  this.jdbc = jdbc;
}

jdbc.update(
  "INSERT INTO USERS (FULL_NAME, EMAIL, PASSWORD_HASH, STATUS) VALUES (?, ?, ?, 'ACTIVE')",
  name, email, passwordHash
);
```

What is happening: Spring injects a JdbcTemplate bean. It uses the configured DataSource/connection pool and the Oracle driver. update() executes an INSERT/UPDATE/DELETE. The question marks are placeholders, and the values are bound as parameters, rather than being concatenated into SQL text.

```
var rows = jdbc.query(
  "SELECT USER_ID, FULL_NAME FROM USERS WHERE EMAIL = ?",
  (rs, rowNum) -> new SessionUser(
    rs.getLong("USER_ID"),
    rs.getString("FULL_NAME"),
    rs.getString("EMAIL")
  ),
  email
);
```

query() is used when multiple rows (or possibly zero rows) may be returned; its RowMapper converts a ResultSet row into a Java object. queryForObject() is used when one value or row is expected. JdbcTemplate handles repetitive JDBC resource management and translates many SQL errors into Spring DataAccessException types.

D. What JDBC URL means

```
jdbc:oracle:thin:@localhost:1521:XE
```

Part	Meaning
jdbc:oracle:thin	Oracle JDBC Thin driver URL prefix.
localhost	Oracle server runs on the same computer.
1521	Listener TCP port shown by lsnrctl status.
XE	Oracle XE 11g SID used by the project.

Do not swap this blindly with jdbc:oracle:thin:@//localhost:1521/XEPDB1. That is a different service-name style often used by newer pluggable-database setups. Your listener showed XE.

6. Oracle data model: four tables

Table	Role and important fields
USERS	USER_ID primary key; FULL_NAME; EMAIL unique; PASSWORD_HASH; STATUS; CREATED_AT.
WALLETS	WALLET_ID primary key; USER_ID foreign key to USERS; CURRENCY_CODE, normally INR; BALANCE NUMBER(14,2); non-negative balance check; unique user/currency.
TRANSACTIONS	TRANSACTION_ID primary key; unique REFERENCE_CODE; TRANSACTION_TYPE; nullable sender/receiver wallet IDs; positive AMOUNT; STATUS; DESCRIPTION; CREATED_AT.
LEDGER	LEDGER_ID primary key; WALLET_ID foreign key; TRANSACTION_ID foreign key; ENTRY_TYPE CREDIT/DEBIT; positive AMOUNT; BALANCE_AFTER.

Relationships

- One USERS row can have one WALLETS row per currency; a unique constraint on (USER_ID, CURRENCY_CODE) prevents duplicate INR wallets for the same user.
- TRANSACTIONS points to sender and/or receiver wallets. For a top-up, sender is NULL and receiver is the credited wallet. For a transfer, both sender and receiver wallets exist.
- LEDGER stores the wallet entries associated with transactions. A transfer creates a DEBIT row for the sender and a CREDIT row for the receiver.

Primary key, foreign key and constraints

- Primary key (PK): unique identifier for each row, e.g. USERS.USER_ID.
- Foreign key (FK): maintains a relationship, e.g. WALLETS.USER_ID references USERS.USER_ID.
- UNIQUE: prevents duplicate values, e.g. email and transaction reference code.

- NOT NULL: field must have a value.
- CHECK: limits valid values, e.g. balance >= 0 and entry type in CREDIT/DEBIT.

Why sequences and triggers?

Oracle XE 11g does not support the later IDENTITY-column feature in the same way newer Oracle releases do, so this schema uses sequences plus BEFORE INSERT triggers to generate primary keys.

```
CREATE SEQUENCE MW_USERS_SEQ START WITH 1 INCREMENT BY 1 NOCACHE;

CREATE OR REPLACE TRIGGER BI_MW_USERS
BEFORE INSERT ON USERS
FOR EACH ROW
WHEN (NEW.USER_ID IS NULL)
BEGIN
   SELECT MW_USERS_SEQ.NEXTVAL INTO :NEW.USER_ID FROM DUAL;
END;
/
```

The sequence produces a new numeric ID. The trigger fires before an inserted row is saved and fills USER_ID only when the insert did not supply one. The legacy MW_ prefix remains on sequence/trigger names; the renamed table itself is USERS.

7. DDL, DML, DQL and transaction control

Category	Meaning	Morrow example
DDL	Defines or changes schema.	CREATE TABLE, CREATE SEQUENCE, CREATE TRIGGER, ALTER TABLE, RENAME table.
DML	Changes rows.	INSERT INTO USERS, UPDATE WALLETS, DELETE FROM LEDGER.
DQL (common teaching term)	Reads rows.	SELECT ... FROM USERS.
TCL	Controls transactions.	COMMIT makes changes permanent; ROLLBACK undoes uncommitted changes.

A PL/SQL block can coordinate several DML statements. In database/02_sample_dml.sql, a demo top-up locks a wallet row, increments BALANCE, inserts TRANSACTIONS, inserts a CREDIT into LEDGER, then commits. On an error, the exception handler rolls back.

8. Authentication flow

Registration: AuthController → AuthService

1. Browser sends POST /api/auth/register with fullName, email and password as JSON.
2. AuthRequests.Register + @Valid validate required name/email/password length. Service trims name and normalizes email to lowercase.
3. BCryptPasswordEncoder(12).encode(password) produces a one-way password hash.
4. Parameterized INSERT adds the row to USERS; a database trigger fills USER_ID. Duplicate email is caught as DuplicateKeyException.
5. Service reads USER_ID and inserts a zero-balance INR row into WALLETS. @Transactional means both inserts succeed together or roll back together.

6. Controller rotates/establishes a server session and returns the new wallet summary. Registration responds HTTP 201 Created.

Login

1. POST /api/auth/login sends email and password.
2. AuthService queries USERS by normalized email and obtains PASSWORD_HASH and STATUS.
3. passwordEncoder.matches(enteredPassword, storedHash) checks the candidate password. The original password is not decrypted or recovered.
4. Only an ACTIVE account with a matching hash succeeds. Invalid credentials return a generic 401-style message.
5. Controller creates a session and calls request.changeSessionId(), then stores the user ID/name/email in that session.

Session details to remember

- The session is kept server-side; the browser holds a session cookie, normally named JSESSIONID.
- application.properties configures HTTP-only and SameSite=Lax cookie behavior, and 30 minutes of inactivity timeout. Secure cookie is set false for local HTTP development.
- Logout invalidates the session. Wallet API controllers require a session and re-check that the user remains ACTIVE.
- This is custom session authentication; the project does not include the full Spring Security web starter. It uses spring-security-crypto for BCrypt.

9. Wallet operations: how the money logic works

Top-up flow

1. Browser POSTs /api/wallet/top-up with amount.
2. WalletRequests validates the amount; WalletService also checks range and scale: minimum ₹1, maximum ₹10,00,000, at most 2 decimals.
3. WalletService finds the user's INR wallet and obtains its row using SELECT ... FOR UPDATE.
4. It calculates the new balance and updates WALLETS.
5. It inserts a TOP_UP row into TRANSACTIONS, then a CREDIT row into LEDGER with BALANCE_AFTER.
6. Because the method has @Transactional, the balance and the two records commit together if successful; a runtime failure should roll them all back.

Important wording: top-up means simulated credit for class demonstration; it does not move real money or confirm a bank payment.

Transfer flow

1. Browser POSTs /api/wallet/transfers with recipientEmail, amount and optional description.
2. Server validates the amount and email, finds an ACTIVE recipient with an INR wallet, and rejects unknown recipients or transfers to oneself.
3. It determines sender and recipient wallet IDs, sorts those IDs, and locks both rows with SELECT ... FOR UPDATE in a consistent order.
4. It checks sender balance >= transfer amount. Insufficient balance returns an error before wallet updates.
5. It subtracts from sender and adds to recipient, updates both WALLETS rows, inserts one TRANSFER row, then two LEDGER rows (DEBIT + CREDIT).

6. All statements are inside one @Transactional method. If any step fails, all related writes roll back; no half-transfer should remain.

Why lock rows?

SELECT ... FOR UPDATE locks selected wallet rows until the database transaction ends. It stops concurrent transfers from both reading the same stale balance and overspending. Sorting wallet IDs before locking also reduces deadlock risk for opposite-direction simultaneous transfers.

Why BigDecimal instead of double?

Money needs decimal arithmetic. BigDecimal avoids common binary floating-point representation errors. The code enforces two decimal places with setScale(2, RoundingMode.UNNECESSARY) and uses NUMBER(14,2) in Oracle.

What is the ledger for?

TRANSACTIONS is the user-facing record of the overall event; LEDGER is the per-wallet accounting record. One transfer has one transaction and two ledger rows: a debit for one wallet and a credit for the other. BALANCE_AFTER captures the wallet balance immediately after that entry.

10. API endpoints: be able to name these

HTTP method + path	What it does	Session?
GET /	Serves index.html via HomeController.	Page route
GET /health	Reports application process is up.	No
GET /health/db	Runs SELECT 'connected' FROM DUAL via JdbcTemplate.	No
POST /api/auth/register	Creates user + INR wallet and signs user in.	Creates session
POST /api/auth/login	Verifies credentials, starts session, returns summary.	Creates session
GET /api/auth/me	Returns current session user summary.	Yes
POST /api/auth/logout	Invalidates session.	Existing session if present
GET /api/wallet/summary	Returns user's wallet balance and totals.	Yes
GET /api/wallet/transactions	Returns recent transactions for that user's wallet.	Yes
POST /api/wallet/top-up	Records simulated credit.	Yes
POST /api/wallet/transfers	Transfers simulated funds to another active user.	Yes

Controller annotations: @RestController means the methods return data (typically JSON), @RequestMapping defines the base route, @GetMapping maps GET, @PostMapping maps POST, @RequestBody turns JSON into a Java request record, and @Valid activates validation annotations.

11. Startup and connection commands

A. Confirm tools

```
java -version
javac -version
mvn -version
lsnrctl status
```

The listener screenshot showed Oracle XE 11g, port 1521 and XE. If `lsnrctl` is unavailable in a terminal, check that Oracle XE is installed and its listener is running.

B. Set environment variables in PowerShell

```
$env:ORACLE_JDBC_URL = 'jdbc:oracle:thin:@localhost:1521:XE'
$env:ORACLE_DB_USER = 'MORROW_APP'
$env:ORACLE_DB_PASSWORD = 'YOUR_CURRENT_MORROW_APP_PASSWORD'
```

Set these in the same terminal session that will run Spring Boot. PowerShell environment variables are per-process/session, not automatically shared with another VS Code terminal. Do not paste the real password into reports or screenshots.

C. Start the backend

```
mvn spring-boot:run "-Dspring-boot.run.arguments=--server.port=8081"
```

Run from the folder containing pom.xml. Port 8081 was chosen to avoid the earlier 8080 conflict. Keep the terminal running while using Morrow. Ctrl+C stops the application.

D. Check health

- http://localhost:8081/health — checks that Spring Boot responds.
- http://localhost:8081/health/db — checks actual JDBC connectivity by querying Oracle's DUAL table.

E. Build and run a JAR

```
mvn test
mvn package
java -jar target/morrow-wallet-0.1.0-SNAPSHOT.jar --server.port=8081
```

The environment variables must also be set in the same terminal used to run the JAR. Maven downloads dependencies from pom.xml, compiles the project, runs tests, and packages the executable artifact.

F. Connect and inspect Oracle

```
sqlplus system@XE
-- enter administrator password when prompted

-- or connect as the application's schema user:
CONNECT MORROW_APP@XE
-- enter MORROW_APP password when prompted

SELECT TABLE_NAME
FROM USER_TABLES
WHERE TABLE_NAME IN ('USERS','WALLETS','TRANSACTIONS','LEDGER')
ORDER BY TABLE_NAME;
```

Use SYSTEM only for database administration such as creating/unlocking MORROW_APP; the web application itself connects as MORROW_APP. SQL scripts referenced as @database/file.sql are easiest to run when SQL*Plus starts in the project root, or by providing a full filesystem path.

12. Live PAT demo sequence

Rehearse this in this exact order. It tells a clear story and proves both input/process/output and database connectivity.

1. Start Oracle XE and its listener. Show `lsnrctl status` only if asked; point out port 1521 and service/SID XE.
2. In one PowerShell terminal, set ORACLE_JDBC_URL, ORACLE_DB_USER and ORACLE_DB_PASSWORD, then start the backend with the Maven command on port 8081.
3. Open /health/db and explain that it runs a test SQL query through JdbcTemplate, the DataSource and Oracle JDBC driver.
4. Open Morrow, register or log in to the first test account.
5. In SQL*Plus, show USERS and WALLETS rows for the two accounts.
6. Perform a simulated top-up through the website. Refresh the SQL queries and show WALLETS, TRANSACTIONS and LEDGER changes.
7. Use the first account to transfer a small amount to the second registered account.
8. Run the queries again. Show sender balance decreases, recipient balance increases, one TRANSFER row appears and two LEDGER rows appear (DEBIT and CREDIT).
9. Try an invalid case, such as insufficient balance or an unknown recipient; explain server-side validation and that rejected requests do not complete a transfer.
10. Explain that data persists because Oracle stores the rows, while money is simulated and no banking network is connected.

SQL queries for the live demo

```
-- Accounts and balances
SELECT U.USER_ID, U.FULL_NAME, U.EMAIL, W.WALLET_ID, W.BALANCE
FROM USERS U JOIN WALLETS W ON W.USER_ID = U.USER_ID
ORDER BY U.USER_ID;

-- Most recent transaction rows
SELECT TRANSACTION_ID, REFERENCE_CODE, TRANSACTION_TYPE,
    SENDER_WALLET_ID, RECEIVER_WALLET_ID, AMOUNT, STATUS, CREATED_AT
FROM TRANSACTIONS
ORDER BY TRANSACTION_ID DESC;

-- Ledger rows
SELECT LEDGER_ID, TRANSACTION_ID, WALLET_ID, ENTRY_TYPE,
    AMOUNT, BALANCE_AFTER, CREATED_AT
FROM LEDGER
ORDER BY LEDGER_ID DESC;
```

13. Common errors and what they mean

Error / symptom	Likely meaning / response
ORA-01017	Incorrect Oracle database username/password. Check environment values and test CONNECT MORROW_APP@XE.
ORA-28000 / LOCKED(TIMED)	Oracle application database account locked after failed logins. Stop Spring Boot, connect as SYSTEM, reset/unlock MORROW_APP, then update app password in the same startup terminal.
ORA-00942	Table/view not found under that schema, wrong connected user/service, or incorrect table name. Check USER_TABLES and use simplified names.

Error / symptom	Likely meaning / response
ORA-12505	Listener did not recognize SID. For this XE 11g setup, confirm XE and port 1521; check `lsnrctl status`.
Port 8080 already in use	Launch Spring Boot with `--server.port=8081`. Do not stop Oracle listener merely to free the web port.
/health works but /health/db fails	Web server runs, but JDBC/Oracle connection or privileges/schema may be wrong.
Login shows generic database error	The UI hides internal DB details. Check latest console stack/error and `/health/db`; verify MORROW_APP is OPEN and can SELECT from USERS.
SQL changes not shown instantly	Website displays what its API last fetched. Refresh or sign out/in so the browser requests fresh summary/history; no push-based live sync is implemented.

14. Viva questions: short answers to practise

Q. What problem does Morrow solve?

A. It demonstrates digital-wallet concepts: user accounts, wallet balances, simulated top-ups, transfers and transaction history.

Q. What is JDBC?

A. Java Database Connectivity, the Java API for executing SQL against a database.

Q. Where is JDBC implemented in this project?

A. The spring-boot-starter-jdbc dependency, the Oracle JDBC driver dependency, datasource settings in application.properties, and JdbcTemplate in AuthService/WalletService/HealthController.

Q. Why JdbcTemplate instead of raw DriverManager?

A. It reduces repetitive connection/statement/result-set handling, uses Spring's DataSource pool, supports parameterized SQL, and translates SQL errors into Spring exceptions.

Q. Do we use ODBC?

A. No. The application is Java and uses JDBC Thin, not ODBC.

Q. Do we use JPA or Hibernate?

A. No. SQL is written explicitly and executed through JdbcTemplate.

Q. What does the JDBC URL mean?

A. jdbc:oracle:thin:@localhost:1521:XE: Oracle Thin driver, server localhost, listener port 1521, XE SID.

Q. Why port 8081?

A. To avoid the previous web-server port 8080 conflict. It is the Spring Boot HTTP port, not the Oracle port.

Q. Difference between port 8081 and port 1521?

A. 8081 serves the website/API; 1521 is Oracle Listener TCP port for database connections.

Q. What is a primary key?

A. A unique non-null row identifier such as USERS.USER_ID.

Q. What is a foreign key?

A. A constraint tying a child row to an existing parent row; e.g. WALLETS.USER_ID references USERS.USER_ID.

Q. Why separate USERS and WALLETS?

A. User identity/authentication data is separate from wallet-specific state, and foreign keys enforce the relationship.

Q. Why TRANSACTIONS and LEDGER?

A. Transactions describe an event; ledger entries describe the per-wallet credit/debit and the resulting balance.

Q. Why sequences and triggers?

A. Oracle XE 11g lacks the newer identity-column feature used by modern Oracle schemas; the sequence generates IDs and trigger assigns NEXTVAL before insert.

Q. What is DDL?

A. Schema-definition SQL like CREATE TABLE, CREATE SEQUENCE and CREATE TRIGGER.

Q. What is DML?

A. Row changes like INSERT, UPDATE and DELETE.

Q. What do COMMIT and ROLLBACK do?

A. COMMIT makes the transaction changes permanent; ROLLBACK undoes uncommitted changes.

Q. How is password security handled?

A. BCryptPasswordEncoder hashes the password on registration. Login compares the candidate with the saved hash using matches(); it does not decrypt the hash.

Q. Where is the password stored?

A. In USERS.PASSWORD_HASH as a BCrypt hash, not plaintext.

Q. How does login persist?

A. The server creates an HTTP session and stores the signed-in user ID/name/email; browser sends the session cookie on same-origin requests.

Q. What happens at logout?

A. AuthController invalidates the current HttpSession.

Q. Why @Transactional?

A. All database changes in the method form one transaction; if a runtime failure happens, Spring rolls back the related writes rather than leaving a partial wallet movement.

Q. Why SELECT FOR UPDATE?

A. Locks the selected wallet row until transaction end, preventing competing operations from changing it based on stale balance data.

Q. Why lock wallet IDs in sorted order?

A. To reduce the chance of deadlocks when two transfers attempt to lock the same pair in opposite order.

Q. Why BigDecimal?

A. It represents decimal monetary values more safely than binary floating-point types and can enforce 2 decimal places.

Q. How do you prevent SQL injection?

A. Use JdbcTemplate SQL placeholders (?) with bound values rather than concatenating user input into SQL text.

Q. How is input validation performed?

A. Bean validation annotations on request records plus explicit service-level checks; DB constraints provide another line of defense.

Q. How do UI changes reflect to SQL?

A. Controller/service executes DML in Oracle; queries in SQL*Plus read the same tables.

Q. How do SQL changes reflect to UI?

A. When the browser requests summary/history again, backend queries Oracle. A refresh or re-login can fetch changed values; it is not push-based realtime synchronization.

Q. Is this real banking?

A. No. Top-ups and transfers are simulated academic operations; no bank or payment network is contacted.

Q. What would you improve for production?

A. Use a supported modern Oracle/driver combination, HTTPS, robust authorization and rate limiting, CSRF strategy, password recovery/verification, audit/monitoring, secrets management, automated integration/security tests and a professional security review.

15. Final pocket cheat sheet

Remember	Answer
One-sentence stack	Java 21 + Spring Boot + Thymeleaf/HTML/CSS/JS + Spring JdbcTemplate + Oracle XE 11g + Maven.
Database connection	jdbc:oracle:thin:@localhost:1521:XE; user MORROW_APP; password supplied through environment variable.
Start command	mvn spring-boot:run "-Dspring-boot.run.arguments=--server.port=8081"
JDBC line to mention	private final JdbcTemplate jdbc; injected through constructor in AuthService/WalletService.
Registration key	BCrypt encode → INSERT USERS → read USER_ID → INSERT WALLETS in @Transactional method.
Login key	SELECT user + PASSWORD_HASH → passwordEncoder.matches() → HttpSession set / rotate ID.
Top-up key	Lock wallet → update balance → insert TOP_UP transaction → insert CREDIT ledger → commit.
Transfer key	Find recipient → lock both wallet rows → check balance → debit/credit wallets → insert transaction + two ledger rows → commit.
Tables	USERS, WALLETS, TRANSACTIONS, LEDGER.
Health endpoints	/health checks app; /health/db checks Oracle using SELECT 'connected' FROM DUAL.
Always be honest	Academic simulation, not a real bank; actual code has controller + service + JdbcTemplate, no separate repository layer.

You do not need to recite every code line. If you can explain the architecture, show the connection settings and JdbcTemplate, walk through one registration and one transfer, and use SQL to prove the database changes, you understand the heart of Morrow.